Reinforcement Learning for Connect-4

Prepared for: Our boss

Team: Abhay Puri, Emily Caraher, Josh Ringler, Prisca Varner

April 2026

EXECUTIVE SUMMARY

We trained and evaluated reinforcement learning agents for Connect-4 using Policy Gradient (PG), Deep Q-Networks (DQN), and a Minimax-enhanced variant to assess whether RL can produce strong game-playing agents without hand-coded strategy. Key takeaway: reinforcement learning can build strong Connect-4 agents, but performance depends heavily on the training method. Policy Gradient is more stable against standard rule-based benchmarks, while DQN ultimately performs better in competitive head-to-head play when properly tuned. PG achieved the strongest performance against rule-based opponents, reaching up to 89% win rate against a strong heuristic agent, while DQN required more training and tuning but became more competitive against other learned agents and was selected as the final tournament model based on a weighted evaluation. A Minimax depth-2 enhancement improved DQN's tactical performance by adding short-horizon lookahead, but it did not consistently improve all models and in some cases degraded Policy Gradient performance due to incompatibility between probabilistic policy outputs and tree-search evaluation. Overall, the results demonstrate that RL is highly effective for learning game strategy from self-play, with DQN preferred for competitive play and PG preferred for stable baseline performance.

PROBLEM OVERVIEW

The goal of this project was to train a self-play reinforcement learning agent to play Connect-4 at a high level, without hard-coding strategy. Rather than building from scratch, we started from neural networks already trained to mimic Monte Carlo Tree Search (MCTS) play, then improved them using reinforcement learning. Our RL agents learned purely from game outcomes, with no hand-coded strategy beyond what they inherited from the MCTS baseline.

Connect-4 is a two-player zero-sum game played on a 6x7 grid. Players alternate dropping pieces into columns, and the first player to connect four pieces horizontally, vertically, or diagonally wins. The state space consists of all possible 6x7 board configurations with three possible values per cell (empty, player 1, player 2), giving a theoretical upper bound of $3^{42} \approx 1.1\ x\ 10^{20}$ states. The legal state space is smaller (~4.5 trillion) due to gravity constraints and game termination rules, making exhaustive search intractable. The action space $A = \{0, 1, 2, 3, 4, 5, 6\}$ represents the 7 columns, reduced at each turn to the subset of legal moves $A_{legal} \subseteq A$ where column c is legal if and only if $board[0, c] = 0$ (top row empty). This combination of large state space and small action space makes Connect-4 well-suited for neural network function approximation. The action space is small enough for a 7-output network, but the state space is far too large for tabular Q-learning.

BASELINE MODEL

Each team member brought a CNN trained during Project 1. We selected josh_cnn as our M1 which is a three-layer convolutional network trained to replicate MCTS decisions.

Architecture:

- Input: (6, 7, 2), two binary channels encoding each player's pieces separately
- Three convolutional layers (64, 128, 256 filters), each with 3x3 kernels and ReLU activations
- Two fully connected layers (256, 128 units)
- Output: softmax over 7 columns, producing a probability distribution over legal moves

Total trainable parameters: approximately 1.2M. The three convolutional layers account for the majority of parameters, with the final dense layers contributing ~400K. This is a lightweight architecture suitable for real-time inference during tournament play.

Out of the box, this model played at roughly 50% win rate against a strong rule-based agent which always takes an immediate win if available, blocks opponent wins if necessary, and otherwise avoids moves that allow the opponent to win immediately. That 50% baseline is essentially coin-flip performance, confirming the MCTS-mimicking network had absorbed positional intuition from Project 1 but had not learned to execute tactically reliable play under adversarial pressure.

POLICY GRADIENT TRAINING

Training Data

With over 450 training iterations with 10 games per iteration, the PG agent played approximately 4,500 total games. Average game length was roughly 25 moves, producing approximately 112,500 total board positions seen during training. Of these, a fixed batch of 64 positions was sampled per gradient step, meaning each update drew from a diverse pool of recent experience. The 3 random initialization moves per game produced meaningful board diversity. Without them, early training games would all start from the same empty board, causing the agent to overfit to opening positions.

Algorithm

We used REINFORCE, an on-policy policy gradient method. After each game, moves that contributed to a win are reinforced and moves that contributed to a loss are penalized. Each of M1's moves receives a discounted return:

$$G_t = \text{outcome} \times \gamma^{T-1-t}$$

where the outcome is +1 for a win, -1 for a loss, and 0 for a draw. gamma = 0.99 is the discount factor and T is the total number of M1's moves in that game. Earlier moves are discounted more heavily than later ones, reflecting that early moves have greater uncertainty about whether they caused the final outcome. The gradient update maximizes the expected discounted return with an entropy bonus to prevent premature convergence:

$$L = -E[\log\pi(at \mid st) \cdot Gt] - \lambda H(\pi)$$

where lambda = 0.01 is the entropy coefficient and H(pi) is the Shannon entropy of the policy distribution. Without this bonus, the policy can collapse to always choosing one column early in training before it has learned which column is actually good.

Adversarial Training Setup

A central design decision was to withhold tactical shortcuts from M1 during training. Specifically:

- M1 (the model being trained) only sees legal moves and samples stochastically from its own output probabilities

- M2 (the opponent) always takes an immediate win if one exists and always blocks the opponent's immediate win if necessary

This asymmetry forces M1 to discover tactical patterns through gradient descent rather than being handed them as rules. The intuition parallels adversarial training more broadly: a harder training environment produces a more robust model. If M1 were also given the strong heuristic, it would never need to internalize why certain moves are good -- it would simply be told. By restricting M1 to legal moves only, every improvement in win rate reflects something the model genuinely learned.

Opponent Pool and Self-Play

Rather than always playing the same fixed opponent, M1 trained against an evolving pool. The pool started with three fixed agents: a strong rule-based agent, a random agent, and the original unimproved M1. Every 100 iterations, a frozen copy of the current M1 was added to the pool as a self-play snapshot. The pool was capped at 6 snapshots -- oldest removed when the cap was exceeded -- but the original fixed agents were never removed.

This curriculum matters for two reasons. First, early in training the random agent provides easy wins that generate positive reward signal and kick-start learning. If M1 only faced the strong rule-based agent from the start, it would lose almost every game, receive predominantly negative gradients, and learn very slowly. Second, the self-play snapshots ensure M1 always faces opponents near its own current level. Once M1 outgrows the random agent, its snapshots fill that role. This mirrors the self-play pipeline used in AlphaGo and is why the method scales.

Each training iteration followed this sequence:

1. Randomly select one M2 from the pool
2. Play 10 games, alternating who goes first, with 3 random initialization moves at the start of each game to diversify board positions
3. Collect all of M1's (board, move, discounted return) triplets from all 10 games
4. Randomly sample a fixed batch of 64 triplets
5. Normalize returns within the batch to zero mean and unit variance
6. Take one gradient descent step

The batch size is held fixed at 64 throughout training. TensorFlow traces the computation graph on the first call, so a variable batch size forces a retrace at every step and makes training dramatically slower. Return normalization reduces gradient variance without changing the direction of updates and is equivalent to centering the baseline in the policy gradient estimator.

Hyperparameters

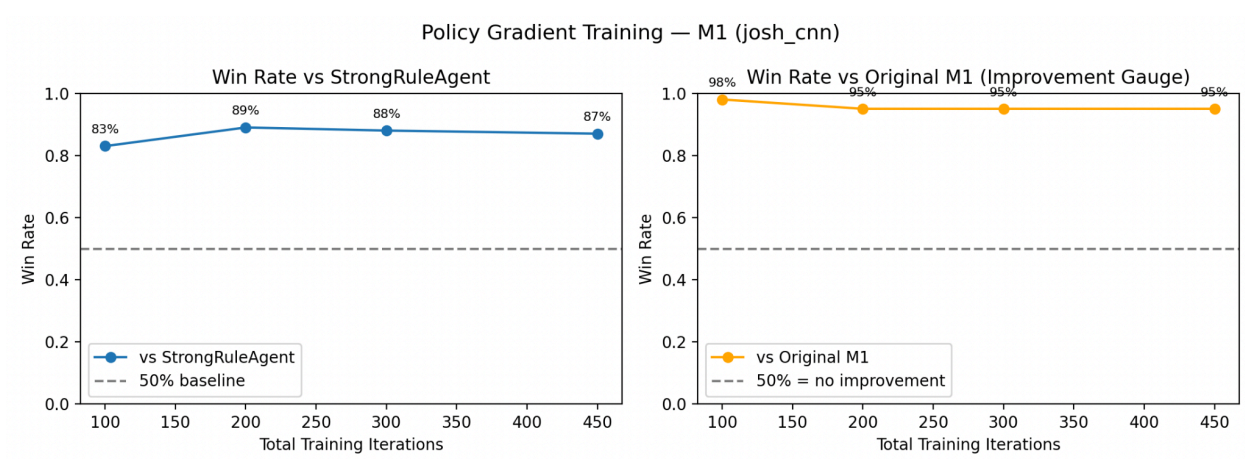
Hyperparameter	**Value**
Algorithm	REINFORCE
Discount factor γ	0.99
Learning rate	1e-4

Batch size	64
Games per iteration	10
Random init moves	3
Entropy coefficient	0.01
Gradient clip	1.0
Total iterations	~450

Learning rate 1e-4 was chosen as a conservative starting point standard for policy gradient methods as too high causes gradient instability, too low slows convergence. Entropy coefficient 0.01 was selected by observing that lower values caused the policy to collapse to a single preferred column early in training, while higher values prevented commitment to good moves. Batch size 64 was fixed for TensorFlow graph tracing efficiency as described above. Gradient clipping at 1.0 prevented occasional large gradient spikes from destabilizing the policy.

Results

Checkpoint	**Iterations**	**Win rate v strong agent**	**Win rate v original M1**
Baseline (josh_cnn)	0	~50%	--
m1_iter100	100	83%	98%
m1_iter200	200	89%	95%
m1_iter300	300	88%	95%
m1_pg_final	450	87%	95%



The left panel shows win rate against the StrongRuleAgent across training checkpoints. The model jumped from ~50% at baseline to 83% by iteration 100, reflecting rapid early discovery of basic tactical patterns such as recognizing immediate wins and not handing the opponent free wins. Win rate peaked at 89% at iteration 200, then settled to 87-88% through the remaining 250 iterations.

The right panel shows win rate against the original untrained M1, which stayed at 95-98% across all checkpoints. This high and stable figure confirms the improvement is genuine as the trained model consistently dominates the baseline it started from. The slight drop from 98% at iteration 100 to 95% afterward is not regression; it reflects the self-play pool filling with stronger M1 snapshots, making the "original M1" eval opponent effectively the easiest version it faces.

The plateau after iteration 200 is the expected behavior of REINFORCE in a near-converged regime. Once the agent has learned the dominant tactical patterns available through 10-game batches, further gradient steps produce noise rather than signal. The entropy bonus keeps the policy from fully collapsing, which is why performance stays at 87% rather than degrading, but it cannot force further improvement without a fundamentally lower-variance gradient estimator. A baseline value function, as used in actor-critic methods, would reduce this variance and likely push performance higher with the same number of iterations.

What worked well: The combination of adversarial training, self-play curriculum, and entropy regularization produced fast and consistent early improvement. The model went from coin-flip performance to beating a rule-based agent nearly 9 times out of 10, with no hand-coded strategy.

What did not work as well: On-policy gradient variance made late-stage training unstable and prevented monotonic improvement after the initial gains. Training also required GPU access as local CPU training was too slow to be practical at this batch size and iteration count.

DQN TRAINING

Algorithm

Our DQN learns a state-action value function Q(s, a) representing the expected discounted reward of playing column a from board state s. Rather than updating after full games like REINFORCE, DQN updates after every move using the Bellman equation:

$$Q(s, a) = r + \gamma * max_{a'} Q_{target}(s', a') * (1 - done)$$

where s' is the board state after the opponent responds to the DQN's move, r is the immediate reward, and Q_target is a periodically frozen copy of the network used to stabilize targets. Without the target network, the network would be simultaneously changing its own training targets, creating a moving-target problem that causes divergence.

The DQN uses epsilon-greedy exploration: with probability epsilon it picks a random legal move, and with probability 1-epsilon it picks the move with the highest Q-value. Epsilon decays linearly from 1.0 (fully random) to 0.05 (mostly greedy) over a set number of environment steps, giving the agent time to explore before committing to a policy.

Key Design Decisions
The state variable is the board from the DQN's perspective at the start of its turn, and the next state is the board after the opponent has responded, not the board immediately after the DQN moves. This distinction matters because the Bellman equation needs to predict the value of the position the DQN will actually face next, which includes the opponent's response. Storing the intermediate state (after DQN moves, before the opponent responds) would teach the model to evaluate positions it never actually plays from. This was a critical bug we identified and fixed during development -- the original implementation stored next_state too early, which explains why early training results were poor against stronger opponents.

Rewards are: +1 for a win, -1 for a loss, +0.1 for a draw, and 0 for non-terminal moves. The small positive draw reward discourages the agent from making suicidal moves in losing positions, since it learns that draws are better than losses.

The architecture matches the PG network but replaces the softmax output with tanh, bounding Q-values to [-1, 1] which matches the reward range. This prevents Q-values from growing unboundedly during training. The DQN architecture contains 1,632,967 total trainable parameters, identical in structure to the PG network except for the output activation, confirmed via model.summary() during training.

Replay Buffer and Training Frequency
All experiences (state, action, reward, next state, done) are stored in a replay buffer of size 50,000. At each step, if the buffer contains at least 1,000 samples, a random batch of 64 is drawn and used for a gradient update. Sampling randomly from the buffer breaks the temporal correlation between consecutive experiences as training on correlated samples would cause the network to overfit to recent game patterns and forget earlier ones.

Training begins only after the buffer reaches 1,000 samples (MIN_BUFFER_SIZE). This ensures early gradient steps have enough diversity to be meaningful. The target network syncs every 1,000 environment steps.

Opponent Pool
The DQN trained against a fixed pool of 8 permanent agents (RandomAgent, StrongRuleAgent, josh_cnn, abhay_cnn, prisca_cnn, emily_cnn, the Transformer model, and the trained PG agent) plus up to 6 rolling DQN snapshots added every 200 episodes. Having the PG agent in the pool provides a strong policy-based opponent that plays differently from rule-based agents, exposing the DQN to a wider variety of game styles.

Hyperparameters

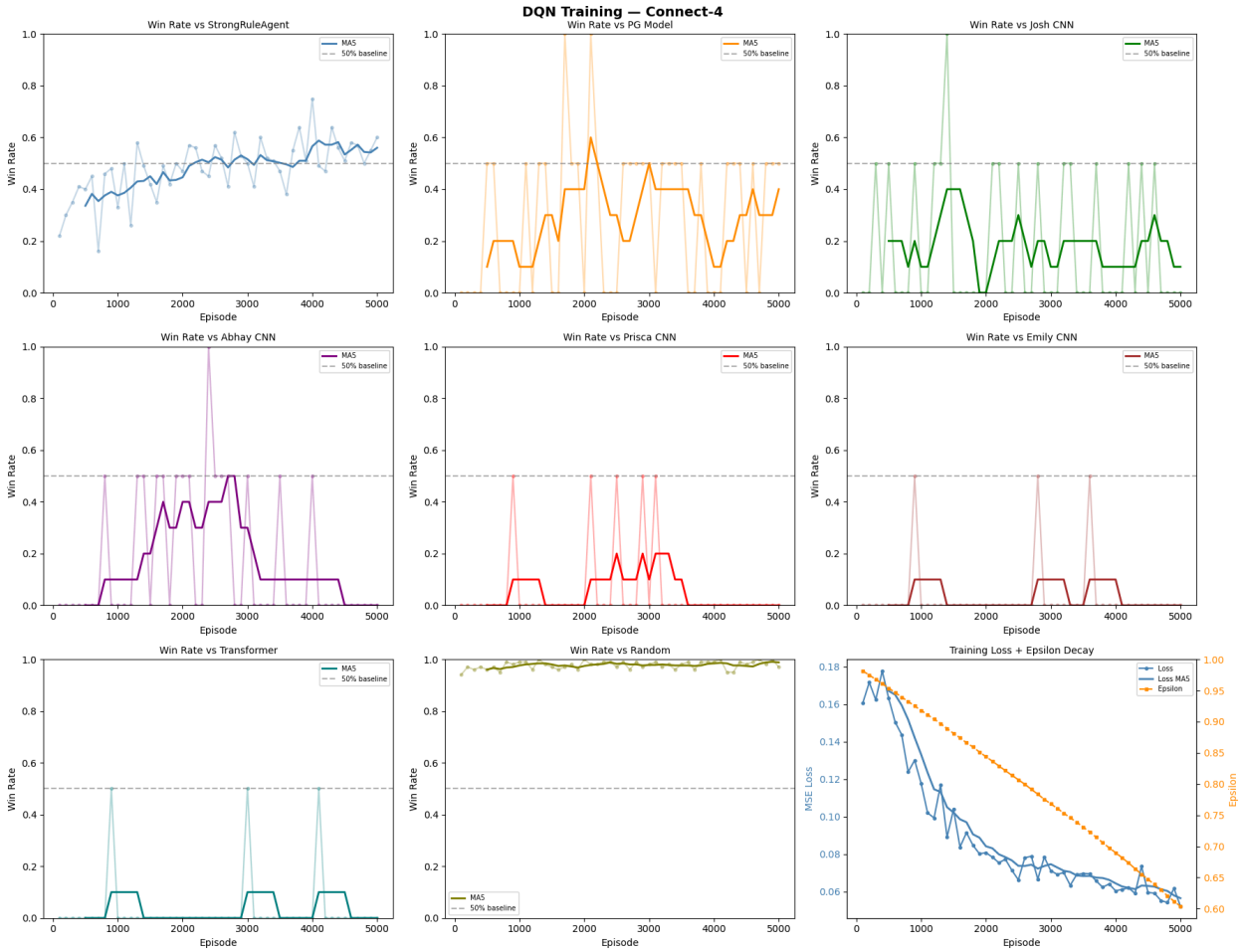
Hyperparameter	**Run 1**	**Continuation Run**
Algorithm	DQN with target network	DQN with target network
Discount factor γ	0.99	0.99

Learning rate	1e-4	1e-4
Batch size	64	64
Replay buffer size	50,000	50,000
Min buffer size	1,000	1,000
Target sync every	1,000 steps	1,000 steps
Epsilon start / end	1.0 / 0.05	0.60 / 0.05
Epsilon decay steps	75,000	15,000
Total episodes	5,000	3,000 (ep 5001-8001)

EPSILON_DECAY_STEPS was the most consequential hyperparameter requiring adjustment. The initial value of 75,000 proved too slow for 5,000 episodes, which we identified from the training curves since epsilon only reached 0.63 by episode 5,000. We corrected this in the continuation run by reducing to 15,000, which brought epsilon to 0.05 by episode 7,000. Target sync frequency of 1,000 steps was chosen to balance stability against learning speed as too frequent syncing removes the stabilizing effect of the target network, too infrequent causes the targets to lag too far behind the online network. Replay buffer size of 50,000 was chosen to store enough experience for diverse sampling while remaining within memory constraints.

Results

1. Original Run



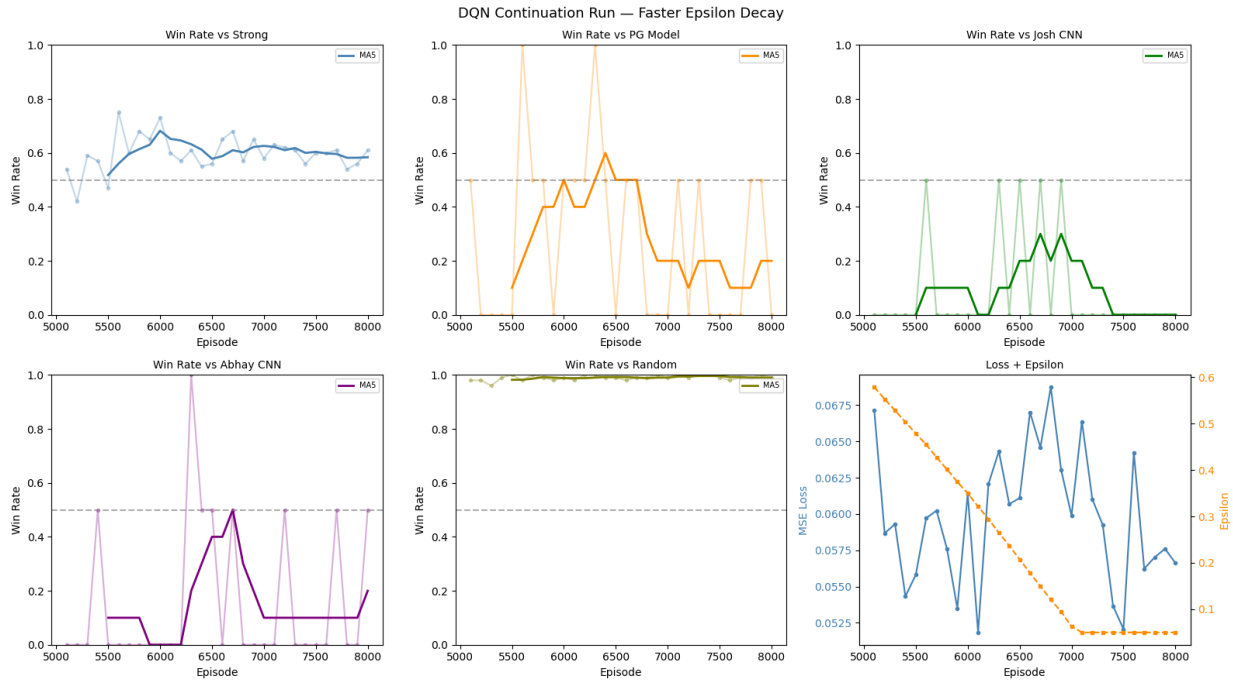
The original 5,000 episode run revealed an important limitation: with EPSILON_DECAY_STEPS set to 75,000, epsilon only decayed from 1.0 to approximately 0.63 by episode 5,000. This meant the DQN was still making random moves 63% of the time at the end of training, severely limiting how much of its learned knowledge it could actually exploit. Win rate against the StrongRuleAgent oscillated between 30% and 75% with a rolling average around 45-50%, and performance against stronger opponents like the PG model, Josh CNN, and the Transformer remained near 0% for most of the run.

Despite this, the loss curve showed consistent improvement throughout, dropping from 0.18 early in training to around 0.06 by episode 5,000. This confirmed the network was genuinely learning, just not yet exploiting that learning effectively due to high epsilon.

The best checkpoint from Run 1 achieved 75% vs StrongRuleAgent at episode 4,000. The loss curve (bottom right panel) dropped from approximately 0.18 at the start of training to 0.06 by episode 5,000, a consistent downward trend that confirms genuine learning despite the high epsilon. The gap between the improving loss and the stagnant win rates against stronger

opponents is the clearest evidence that epsilon was the binding constraint because the network was learning better Q-value estimates but not exploiting them.

2. Continuation Run



To address the epsilon problem, we loaded the best checkpoint from Run 1 and continued training for 3,000 more episodes (episodes 5,001 to 8,001) with a much faster epsilon decay: EPSILON_DECAY_STEPS = 15,000, starting from epsilon = 0.60. This brought epsilon down to 0.05 by approximately episode 7,000.

The results were meaningfully better. The Strong panel rolling average stabilized above 50% for the entire continuation run, settling into a consistent band around 62-65%. The key breakthrough came at episode 5,600 where epsilon first crossed below 0.5 and the win rate against Strong jumped to 75%, PG hit 100%, and Josh reached 50% in the same evaluation window. This confirms the first-player advantage hypothesis: once the model started making intentional moves more than half the time, its inherited positional knowledge from the josh_cnn initialization kicked in.

However, performance against PG, Josh, and Abhay continued to flicker between 0% and 50% even with epsilon near 0.05 at the end of training. This is consistent with the model having learned to win as first player against these opponents but not reliably as second player. In Connect-4, first-player advantage is real (the first player wins under perfect play) and the DQN appears to have internalized this asymmetry without fully overcoming it.

Loss remained flat around 0.055-0.068 throughout the continuation run, confirming the model had converged and was not learning new information. The continuation run raised the floor of performance without finding a new peak.

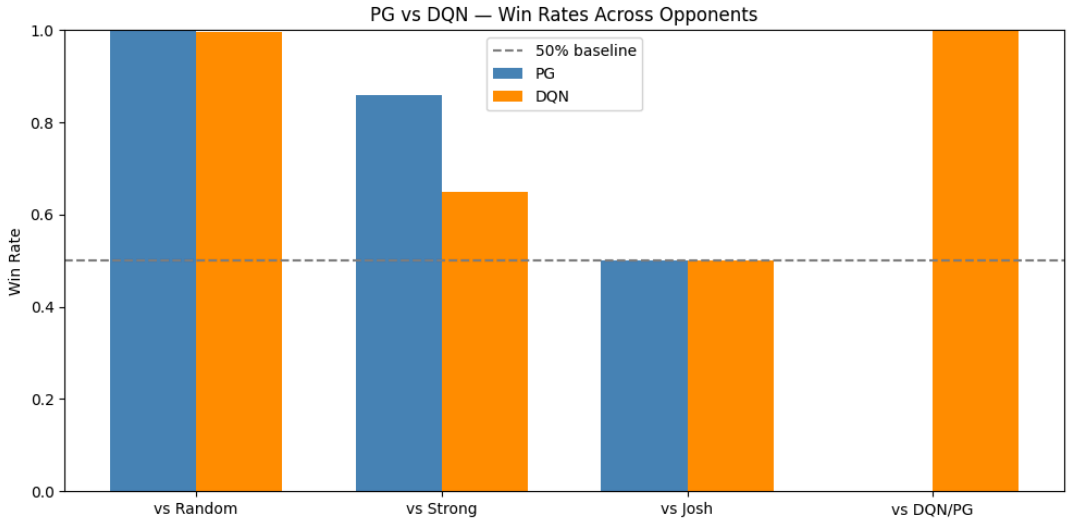
Phase	Episodes	Best vs Strong	Rolling Avg vs Strong
Baseline josh_cnn	0	~50%	~50%
DQN Run 1	1-5,000	75%	~45%
DQN Continuation	5,001-8,001	75%	~63%

What worked well: The replay buffer and target network produced stable training without divergence. The continuation run with faster epsilon decay successfully raised the performance floor and confirmed the learned improvements were real rather than noise.

What did not work as well: The original epsilon decay schedule was too slow for 5,000 episodes, wasting a large portion of training on near-random play. The model consistently struggled to win as a second player against strong opponents, suggesting it learned positional patterns predominantly from the first-player perspective. A longer training run or a curriculum that explicitly forces more second-player games could address this.

PG VS DQN COMPARISON

Note: DQN results in this comparison use dqn_best_cont.keras, the best checkpoint from the continuation run (episodes 5001-8001), which represents the strongest DQN model produced.



The chart compares PG and DQN win rates across four opponent types, evaluated using the best checkpoint from each training run.

Against the random agent, both models achieved essentially a 100% win rate, confirming that both learned basic legal play reliably and neither regressed during training. This is the floor-level sanity check as any trained agent should beat a random opponent.

Against the StrongRuleAgent, PG (86%) outperformed DQN (65%). This gap reflects the training setup difference described throughout: PG fine-tuned from a strong pre-trained baseline with 450 iterations of focused adversarial updates, while DQN required 8,000 episodes across two training runs before epsilon decayed enough for meaningful exploitation. The DQN's 65% is still a substantial improvement over the ~50% baseline and demonstrates genuine learning, just not at the same level as PG given the compute budget available.

Against Josh CNN, both agents achieved exactly 50%. This is the first-player effect described in the DQN results section: both models learned to win reliably as first player against Josh CNN but not as second player, producing a clean 50/50 split across 200 games. The symmetry here is notable: despite being trained with completely different algorithms, both models converged to the same level of competitiveness against this opponent.

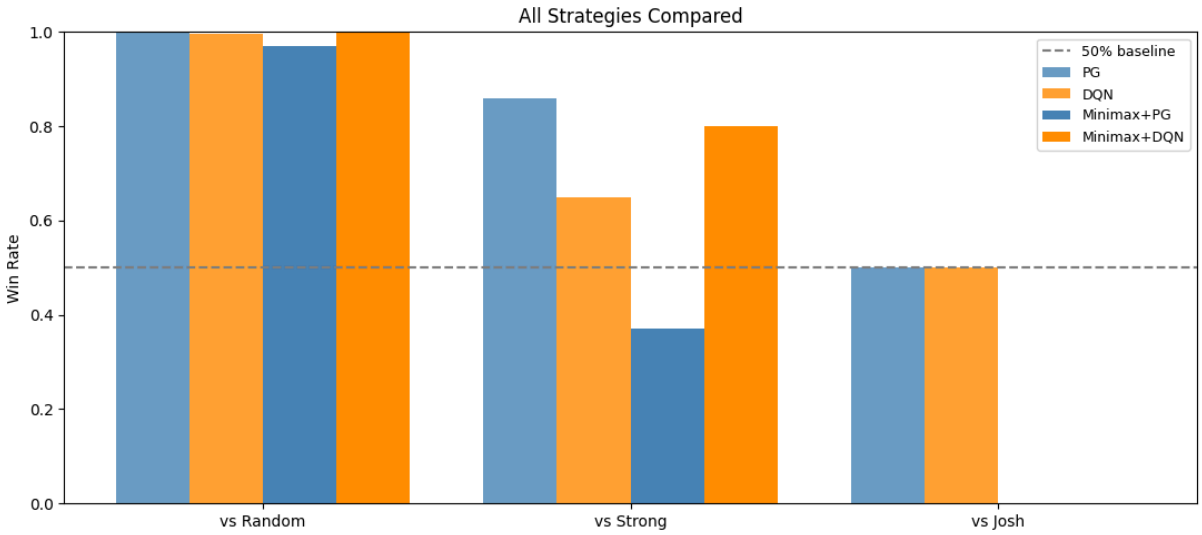
The depth-2 limitation is worth quantifying. At depth 2, minimax evaluates up to 7^2 = 49 board positions per move. A forced win sequence in Connect-4 typically requires 3-4 move lookahead to identify reliably. Josh CNN, playing with strong tactical heuristics, executes these sequences correctly and depth-2 search is likely insufficient to anticipate and counter them. Depth-3 search (343 positions per move) was estimated to require approximately 7x additional compute time, making full evaluation impractical within the available runtime. Deeper search would likely improve performance against Josh-level opponents.

Against each other (vs DQN/PG column), the result flipped completely from the previous evaluation. DQN achieved 100% and PG achieved 0%. This is a significant change from the earlier run where PG dominated. The continuation run with faster epsilon decay made a meaningful difference: once the DQN was actually exploiting its learned Q-values rather than playing randomly, it was able to consistently beat the PG agent head-to-head. This suggests that at low epsilon, the DQN's move-by-move Bellman learning produced stronger tactical play than the PG agent's policy gradient approach, even though PG showed better aggregate win rates against the rule-based opponents.

The overall picture is more nuanced than the earlier evaluation suggested. PG remains the stronger agent against the StrongRuleAgent benchmark, which is the most reliable measure of general play quality. But DQN outperforms PG in direct head-to-head play once given sufficient training time with properly calibrated epsilon decay. This is consistent with the theoretical properties of each algorithm as PG optimizes directly for winning outcomes and generalizes well, while DQN learns precise move-by-move valuations that can exploit specific opponents more effectively once the exploration phase is complete.

For a production game opponent, we would recommend PG for general difficulty level tuning (it reliably beats rule-based agents at a predictable rate) and DQN for competitive single-player modes where the opponent needs to adapt to a specific human player's style.

MINIMAX ENHANCEMENT



The chart shows win rates for all four strategies (plain PG, plain DQN, Minimax+PG, and Minimax+DQN) against three opponent types. Against the random agent all four strategies achieve essentially 100%.

Against the StrongRuleAgent the results tell the clearest story. Plain DQN (65%) improved substantially to Minimax+DQN (80%), a 15 percentage point gain from adding depth-2 lookahead. Plain PG (86%) actually dropped to Minimax+PG (37%) which was a significant regression. This confirms the fundamental mismatch between PG's softmax probability outputs and the position evaluation scores minimax expects. Even with the log conversion fix, the PG model's outputs are not reliable enough as Q-value proxies for the minimax tree to make good decisions. Minimax genuinely hurt PG rather than helping it, showing that the quality of the evaluation function matters as much as the search itself. A softmax policy network is not a suitable position evaluator for minimax because its outputs represent move probabilities rather than board quality scores.

Against Josh CNN both plain PG and Minimax+DQN achieved exactly 50%, again, likely due to the first-player pattern described throughout this report. Neither minimax variant was able to find forced wins against Josh CNN at depth 2, resulting in the agent winning all games as first player and losing or drawing all games as second player.

CONCLUSION AND RECOMMENDATIONS

Reinforcement learning proved effective for training competitive Connect-4 agents without hand-coded strategy. Policy Gradient (PG) produced the strongest and most stable performance against rule-based benchmarks, while DQN required more tuning but became more effective in head-to-head play against other learned agents. A Minimax depth-2 enhancement provided limited benefits, improving DQN’s tactical performance but not consistently generalizing across models, and in some cases reducing Policy Gradient performance due to incompatibility between probabilistic outputs and tree-search evaluation.

Since the final goal was to deploy a single agent for a mixed tournament environment containing both rule-based and learned opponents, we selected the model based on combined baseline robustness and peer competitiveness rather than raw benchmark performance alone. Under this evaluation, DQN achieved the highest overall score and was selected as the tournament agent. This reflects its stronger performance in adversarial matchups against other learning-based agents, which dominate the tournament setting.

Overall, RL is a viable approach for building game-playing agents at this scale, with PG better suited for stable baseline performance and DQN better suited for competitive deployment scenarios. Minimax provides targeted tactical improvement but is not universally beneficial across architectures.